

PROJECT AND TEAM INFORMATION

Project Title

Smart Emergency Response System

Student / Team Information

<i>Team Name:</i>	<i>ALPHA</i>
Team member 1 (Team Lead)	<i>Pandey, Sambhawi – 2510380216 2510380216@geu.ac.in</i> 
Team member 2	<i>Rawat, Atharv – 2510370380 2510370380@geu.ac.in</i> 
Team member 3	<i>Lalotra, Aayush – 2510012608 2510012608@geu.ac.in</i> 
Team member 4	<i>Garg, Shivam – 2510386512 2510386512@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

In emergencies such as road accidents, fires and medical emergencies, the need for a fast and efficient response is high. In a city, there can be more than one emergency happening concurrently and a limited number of ambulances, emergency vehicles and response teams available. If decisions are made manually, deciding which emergency should be handled first, which resource should be assigned, and which route should be followed may become difficult in such cases. The proposed Smart Emergency Response System is to solve this problem by developing an algorithm based system that assists in prioritizing emergencies and efficiently allocating available emergency resources. The city road network will be represented as a graph, where locations are vertices and roads are weighted edges. We will use Dijkstra's shortest-path algorithm to find an optimal path between points. The city road network will be represented as a graph, where locations are vertices and roads are weighted edges. We will use Dijkstra's shortest-path algorithm to find an optimal path between points. In case of emergencies we will use a priority queue, which will prioritize the emergencies based on their severity. Queues, searching, sorting, linked-lists/vectors, and hashing can also be added for efficient data handling. The project will be implemented in C++ in Object Oriented Programming where classes will be used to represent things like emergencies, ambulances, hospitals and locations. The first system will be a prototype based on simulation to demonstrate how DSA and OOP can be used to solve a real-world problem

State of the Art / Current solution

Today, emergency response usually means an emergency call center, dispatchers, ambulance services, police, fire and hospitals all working together to respond to incidents. Emergency information is obtained. The incident is evaluated and an available response unit is sent to the location. Modern emergency-management solutions may also utilize digital maps, GPS, communication systems and location-based services to assist with dispatch and navigation. But the effectiveness of the response could still be impacted by such issues as availability of resources, traffic conditions, communication between agencies, and the decisions made during dispatch. The proposed project is not intended to replace existing emergency services. It's more about the algorithmic decision making part of emergency response. The system will give a simulation for prioritizing emergencies, identifying the available resources and computing the routes using DSA. The project addresses the key gap in the development of an integrated educational prototype that demonstrates how priority queues, graphs, shortest-path algorithms, searching, sorting and OOP can work together to support emergency-response decisions.

Project Goals and Milestones

The primary objective of the project is to design a C++ based Smart Emergency Response System that will efficiently handle the emergency requests by ranking the incidents, allocating the available emergency resources and finding the suitable routes using DSA and OOP concepts.

Major Goals :

1. Develop an emergency-reporting and management module.
2. Categorize emergencies according to severity.
3. Implement a priority queue/heap for emergency prioritization.
4. Represent the city road network using a graph.
5. Implement Dijkstra's algorithm for shortest-path calculation.
6. Manage ambulances and other emergency resources.
7. Use appropriate searching and sorting techniques for efficient data management.
8. Apply OOP concepts through modular classes and objects.
9. Build a menu-driven C++ prototype.
10. Test the system using different emergency and resource scenarios.

Initial Milestones

Milestone 1: System design and requirement analysis

Milestone 2: Design classes for Emergency, Ambulance, Hospital, Location and Response System

Milestone 3: Emergency management with queues and priority queues

Milestone 4: Cities as a graph

Milestone 5: Implement Dijkstra's Shortest Path Algorithm

Milestone 6: Integration of resource allocation and route selection

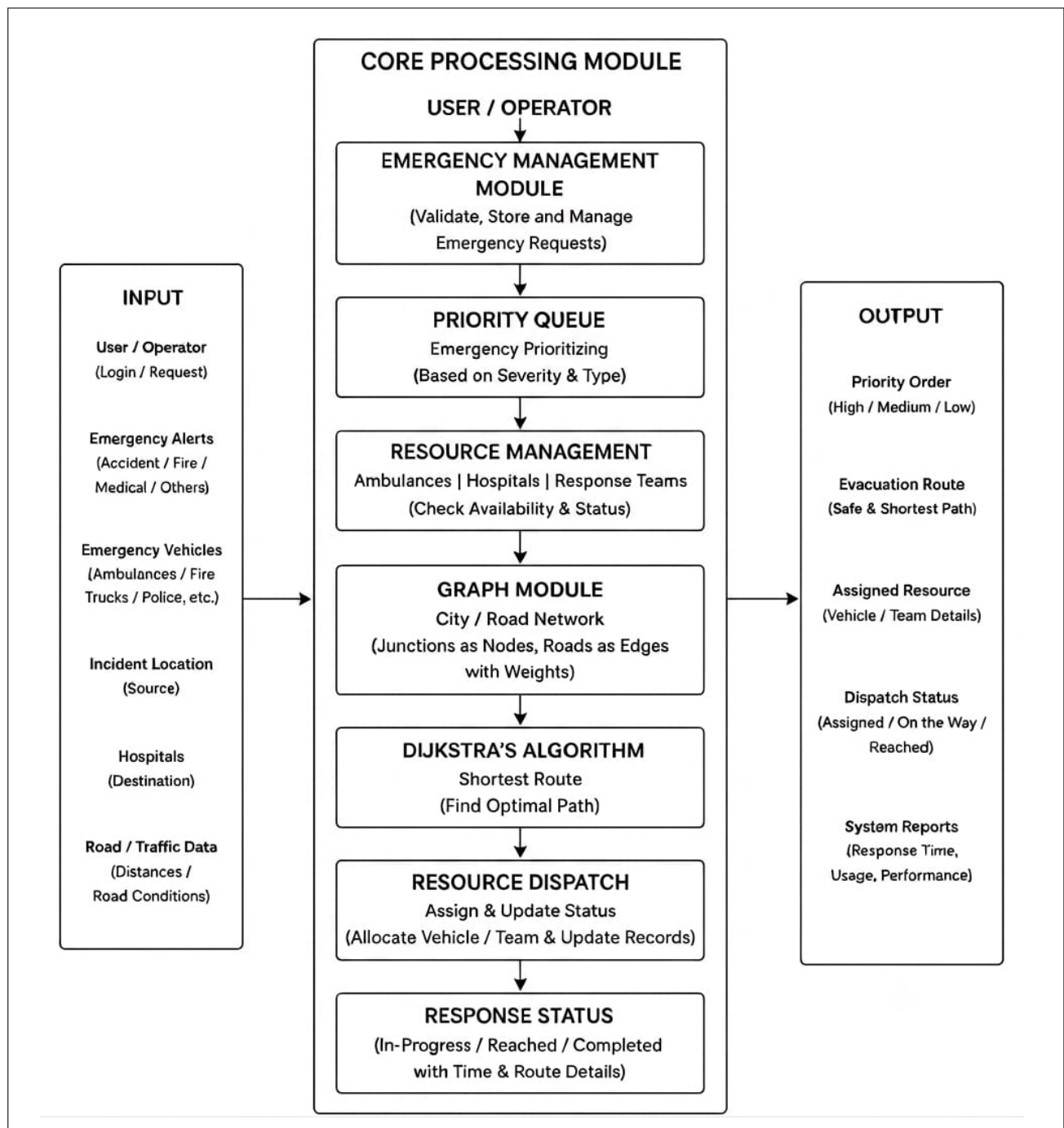
Milestone 7: Finish C++ prototype with menu-driven interface

Milestone 8: Testing, debugging, performance analysis and final documentation

Project Approach

The project will be developed incrementally and modularly. First, the requirements of an emergency-response system will be examined and the major entities and operations will be identified. Then the system will be designed using Object Oriented Programming in C++. The major components will be represented by classes like Emergency, Ambulance, Hospital, Location, and EmergencyResponseSystem. Encapsulation will be used to protect data, and proper OOP principles will be used to make the system modular and maintainable. We will select different concepts in DSA based on their practicality. Queue will be used to process incoming emergency requests. Priority queue/heap will sort emergencies by severity. The city will be a weighted graph. Locations will be vertices, roads will be edges. Dijkstra's algorithm will find the shortest path from the emergency location to the available response resources. Techniques for searching and sorting will be employed to locate and arrange emergency resources. Dynamic collections can be implemented using linked lists/vectors and hashing can be used to speed up resource lookup by unique IDs. The first platform will be a C++ console-based application developed and tested with a suitable C++ development environment like Visual Studio Code. The system will use simulated data for emergencies, roads, ambulances, and hospitals. The development process will be incremental, where individual data structures and algorithms will be developed and tested independently before inclusion in the full emergency-response workflow.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The main outcome of the project will be a working C++ console-based Smart Emergency Response System that demonstrates how Data Structures and Algorithms (DSA) and Object-Oriented Programming (OOP) can be applied to manage emergency situations efficiently.

The system will include the following features:

- 1. Registration and management of emergency requests.*
- 2. Classification and prioritization of emergencies based on their severity.*
- 3. Management of available ambulances and other emergency resources.*
- 4. Representation of the city road network using a weighted graph.*
- 5. Calculation of the shortest route using Dijkstra's algorithm.*
- 6. Selection and assignment of a suitable available emergency resource.*
- 7. Updating the status of emergencies and assigned resources.*
- 8. Displaying relevant emergency, resource, and route information.*
- 9. Efficient organization and retrieval of data using suitable searching and sorting techniques.*

Expected Deliverables :

The project will include the following deliverables:

- 1. C++ source code.*
- 2. A working menu-driven prototype.*
- 3. System architecture diagram.*
- 4. Test cases along with their results.*
- 5. Project report.*
- 6. Presentation/PPT and project demonstration.*

Assumptions

The initial system will operate as a simulation using predefined city locations, roads, emergency requests, ambulances, and hospitals. Road distances or travel costs will be represented using numerical weights. Ambulance and hospital availability will be provided as simulated data rather than obtained from real emergency departments. The system assumes that emergency severity can be represented using predefined priority levels. Real-time GPS, traffic information, external APIs, and live hospital data are outside the initial third-semester scope and may be considered for future development

References

- 1. C++ Reference — cppreference.com*
- 2. Dijkstra's Algorithm — CP-Algorithms*
- 3. Class notes and reference material provided by the faculty*